Interactivity & Navigation

User Interface is all the elements on the screen (images, colors, panels, text, etc) and the *User Experience* is what happens when the user's interact with the interface. "UI is the what and UX is the how"
A stateful widget is divided into two classes: the widget and its state. Every stateful widget has a state object to help manage its life cycle (a completely separate class). As a design point, `StatefulWidgets` are of the few scenarios where two classes (the widget and the widget's state) should be in the same file. In `StatefulWidgets`, you put the `build` method in the `State` class, not in the widget

lib > stopwatch.dart > ...

```
import 'package:flutter/material.dart';
import 'dart:async';

class StopWatch extends StatefulWidget {
  const StopWatch({super.key});
  @override
  State<StopWatch> createState() => _StopWatchState();
}

class _StopWatchState extends State<StopWatch> {
  int seconds = 0;
  late Timer timer;
  @override
  void initState() {
    super.initState();
    timer = Timer.periodic(const Duration(seconds: 1), _onTick);
  }

  void _onTick(Timer time) {
    if (mounted) {
      setState(() {
        ++seconds;
      });
    }
  }

  String _secondsText() => seconds == 1 ? 'second' : 'seconds';
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Stopwatch'),
      ), // AppBar
      body: Center(
        child: Text(
          '$seconds ${_secondsText()}',
          style: Theme.of(context).textTheme.headlineMedium,
        ), // Text
      ), // Center
    ); // Scaffold
  }

  @override
  void dispose() {
    timer.cancel();
    super.dispose();
  }
}

```

Remember, `StatefulWidgets` are made up of two classes: **the widget** and

its state. The widget part of the `StatefulWidget` does not usually do much because the data that changes is placed in the `State` class. **All widgets are immutable**, but the `State` object is mutable. The `State` object is responsible for the `build` method. *Jargon*, "dirty state" means the in the next frame it will repaint. The `setState` function tells Flutter that a widget needs to be repainted. Every time `setState` is called the `build` method is called. Never call `build` directly, rather do it through `setState`.
The following diagram summarizes how Flutter's render loop is impacted by `setState`

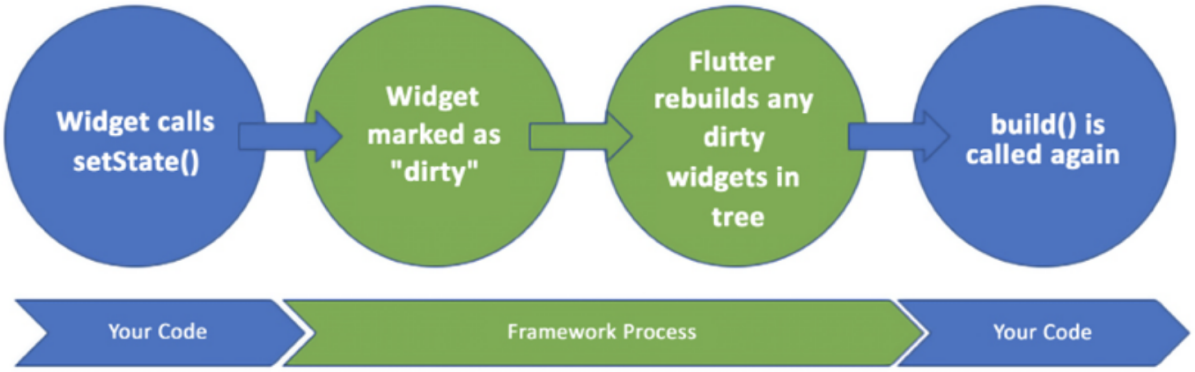

Figure 6.2: setState and build in Flutter

Note, that complex operations should not be performed in the `setState` function, but just the final value assignment.

`State` Life Cycle

Unlike `StatelessWidget`, which has nothing more than a `build` method. `StatefulWidgets` have a life cycle of methods that are called in a specific order:

- `initState`
- `didChangeDependencies`
- `didUpdateWidget`
- `build` **(required)**
- `reassemble`
- `deactivate`

- `dispose`

`initState` >

- used to initialize any non-final state in the class
- *kinda like a constructor*
- This method is called **after** the constructor but **before** it is added to the widget tree.

`didChangeDependencies` >

- this method is called after `initState`, but has access to the `BuildContexton`
- Useful for any set up work that requires `context`

`build` >

- **required**
- Identical to the `StatelessWidget` version
- This defines and returns the widget's tree (creating the UI)

`dispose` >

- The cleanup method
- Called when the `State` object is removed from the widget tree
- One must explicitly release finished resources

Buttons

Flutter has several aesthetically different buttons

- `ElevatedButton`
- `TextButton`
- `IconButton`
- `FloatingActionButton`
- `OutlinedButton`
- `DropDownButton`
- `CupertinoButton`

Scrolling

`ListView` has scrolling builtin by default when its widgets extend past the screen.
A scrolling widget needs to know their parent's constraints to activate scrolling, putting scroll widgets inside widgets with unbounded constraints can cause Flutter to throw errors.

Constraints go down -> Sizes go up

More in Layout
In Flutter there's a rule: "Constraints go down. Sizes go up.". This means that:

- Parent widgets pass down constraints to their children. Constraints are passed in a `BoxContraints` object, that includes minimum and maximum values for the width and height of the children.
- Children widgets determine their size and then pass the information back up to the parent widget
  If you place a scrolling widget inside a `Flex` widget
  (like `Row` or `Column`), which is pretty common, just don't forget to always wrap the scrolling content in `Expanded` or `Flexible` first.

This error means that you have an unbounded scrolling widget

```
I/flutter (28416): ══╡ EXCEPTION CAUGHT BY RENDERING LIBRARY ╞═══════════════════════════
I/flutter (28416): The following assertion was thrown during performResize():
I/flutter (28416): Vertical viewport was given unbounded height.
I/flutter (28416): Viewports expand in the scrolling direction to fill their container.In this case, a vertical
I/flutter (28416): viewport was given an unlimited amount of vertical space in which to expand. This situation
I/flutter (28416): typically happens when a scrollable widget is nested inside another scrollable widget.
I/flutter (28416): If this widget is always nested in a scrollable widget there is no need to use a viewport because
I/flutter (28416): there will always be enough vertical space for the children. In this case, consider using a Column
I/flutter (28416): instead. Otherwise, consider using the "shrinkWrap" property (or a ShrinkWrappingViewport) to size
I/flutter (28416): the height of the viewport to the sum of the heights of its children.
I/flutter (28416):
I/flutter (28416): When the exception was thrown, this was the stack:
I/flutter (28416): #0      RenderViewport.performResize.<anonymous closure> (package:flutter/src/rendering/viewport.dart:1147:15)
I/flutter (28416): #1      RenderViewport.performResize (package:flutter/src/rendering/viewport.dart:1200:6)
I/flutter (28416): #2      RenderObject.layout (package:flutter/src/rendering/object.dart:1604:9)
```

Handling Large Datasets with `ListView` (Scrolling)

The problem is handling large amount of data in a scrolling widget. We know only a certain amounts of views (widgets that hold data) fit on any screen at once. The trick is to, recycling views with new data instead of creating new views.

The `itemCount` property to the `ListView.builder()` lets Flutter know how long the list is. duh!

The `itemBuilder` property and its value (a closure) help with performance to limit the creation of `Elements` and `RenderObjects`. `itemBuilder` is using *deferred rendering* to create only what it needs and manage a subset of the total widgets. The *scroll tree* is limited to only what is needed.

Flutter will continuously call `itemBuilder` with an updated index as the user scrolls. Assuming the size of the viewport is constant, the programmer only manages the fixed number of visible entries.

Figure 6.8: Scrolling Viewport

`itemExtent` property is a way to supply a fixed height to all the items in `ListView`. Instead of letting the widget figure out its own height based on the content, using `itemExtent` will enforce a fixed height and leads to performance benefits.

`ScrollController` object allows programmers to interact with a `ListView` from outside the build methods.

```dart
scrollController.animateTo(
itemHeight * laps.length,
duration: Duration(milliseconds: 500),
curve: Curves.easeIn,
);
```

```
`itemHeight * laps.length,` this argument dictates where to
scroll to.
Since, there we set items to have a fixed height we can
calculate how far to automatically scroll.

`duration: Duration(milliseconds: 500),` dictates how long it
will take to scroll to the calculated position.

`curve: Curves.easeIn,` how (animate) to get there

### Working with `TextField`s
`TextField` has a subclass called `TextFormField` for text
fields that interact with each other. `TextField`s are
platform aware widgets.
`validator` on failure will return a string and on success
return `null`.
`Form` widget can wrap many `TextFields` and is a non-
rendering widget. `Form` can contain non-`TextFields`, but
keeps track which ones `null`(valid). If all validator
functions wrapped in `Form` return `null` then the entire
`Form` is valid.
`GlobalKey` used to get access to the form’s state class from
```

```
outside the `build` method. `GlobalKey`'s uniquely identify
elements. While `BuildContext` is an object that can find
**parents**  in the widget tree, `GlobalKey`'s are objects
that you use to retrieve a **child** widget tree. (_GlobalKey
is a complex topic. This is a brief, crude intro_). In short,
with the key, you can retrieve the Form’s state.
The `FormState` class has a public method
called `validate` that will call the validator on all its
children.
### Navigating to the next screen
**Now working from stopwatch part 2 project**
Within `MaterialApp` a class called `Navigator` manages
screens. The `Navigator` class abstracts screens to the
concept of `Routes`.
Regarding a login event, once a use logs successfully the
back button to return to the login screen should be
inaccessible.
```dart
Navigator.of(context).pushReplacement(
        MaterialPageRoute(
                builder: (_) =>
                        StopWatch(name: name, email: email),
), );
```
```

`Navigator`'s function as a stack. `Routes` can be pushed onto and popped off of.The above snippet will remove the `Route` (screen) from the `Navigator` stack.
`pushNamed` does pop then push of the current stack

Presenting Bottom Sheet

Now working from stopwatch part 3 project

A bottom sheet is like a footer that be animated. It can temporarily or

permanently appear after an event.
The app's widget looks like.

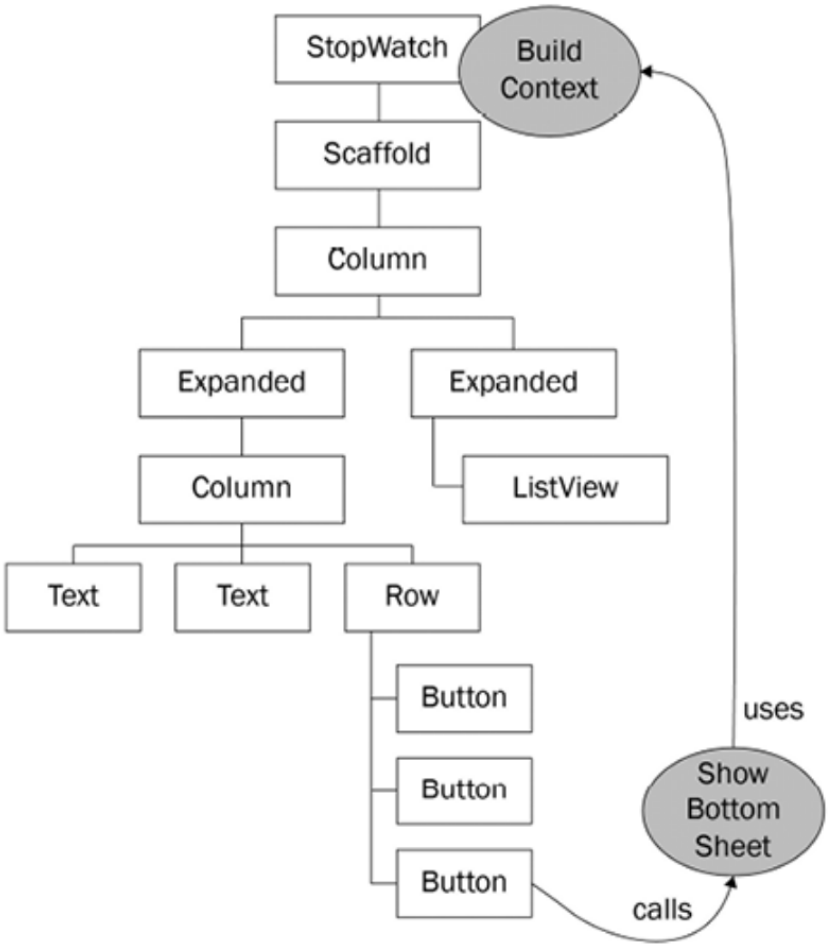

Figure 6.13: Widget tree and BuildContext

Bottom sheets are partial routes. They use the closest `Scaffold` by using the `of-context` to find it. The above graphic shows the `BuildContext` we do **not** want. The error message `No Scaffold Widget found` will be triggered when the nearest `Scaffold` is too far up in the tree to be used. The solution is to use a `BuildContext` lower in the tree. Wrapping the button (that uses the bottom sheet) in the `Builder` widget will solve this.

```
Builder(builder: (context) {
  return TextButton(
    style: ButtonStyle(
      backgroundColor:
          MaterialStateProperty.all<Color>(Colors.red),
      foregroundColor:
          MaterialStateProperty.all<Color>(Colors.white),
    ), // ButtonStyle
    onPressed: isTicking ? () => stopTimer(context) : null,
    child: const Text('Stop'),
  ); // TextButton
}), // Builder
```

The above snippet demonstrate how to create a `BuildContext` closer to the button.

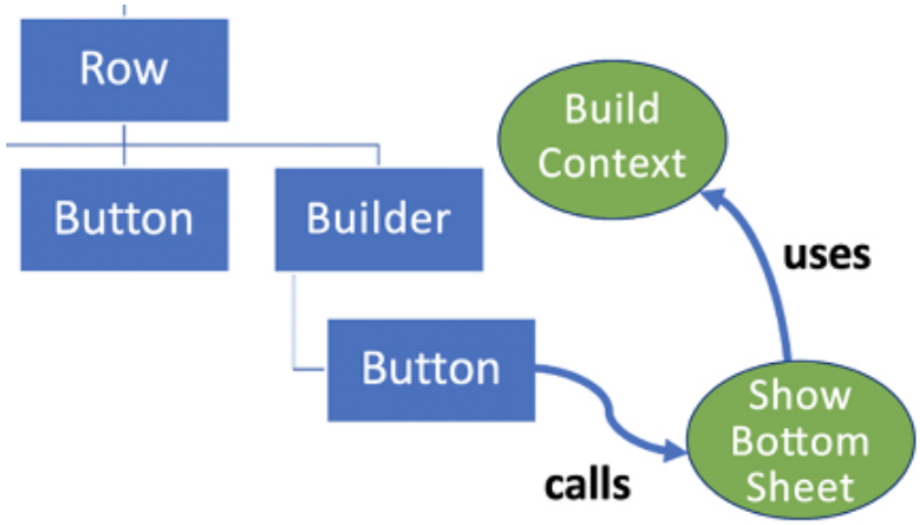

Figure 6.14: New widget tree with a Builder

Persistent controllers

```
final controller =
    showBottomSheet(context: context, builder: buildRunCompleteSheet);
Future.delayed(const Duration(seconds: 5)).then((_) {
  controller.close();
});
```

The `showBottomSheet` method returns a **persistent controller** used to manipulate them.